

LAPORAN KEMAJUAN

PENGEMBANGAN SISTEM ELECTRONIC NOSE (E-NOSE) KOPI BERBASIS ESP32-S3 DENGAN KONEKTIVITAS IoT GANDA DAN PEMBARUAN FIRMWARE OVER-THE-AIR

Mata Kuliah: Teknologi IoT

Dosen: Ahmad Radhy, S.Si., M.Si.



Disusun oleh:

Ahmadin Fatkhurahman 2042241015

Revi Azizu Rohman 2042241029

DEPARTEMEN TEKNIK INSTRUMENTASI
FAKULTAS VOKASI
INSTITUT TEKNOLOGI SEPULUH NOPEMBER

2026

Daftar Isi

1	Pendahuluan	3
1.1	Latar Belakang	3
1.2	Tujuan Proyek	3
1.3	Ruang Lingkup Laporan	3
2	Arsitektur Sistem	4
2.1	Gambaran Umum	4
2.2	Perangkat Keras	4
2.3	Konektivitas	5
2.4	Firmware	5
3	Progres yang Telah Dicapai	6
3.1	Prototipe Perangkat Keras	6
3.2	Akuisisi Sensor	7
3.3	Konektivitas WiFi dan MQTT Ganda	7
3.4	Aplikasi GUI Desktop	7
3.5	Kerangka TinyML (Model Contoh)	8
3.6	Pembaruan Firmware Over-The-Air (OTA)	8
3.6.1	Persiapan: Tabel Partisi Ganda	8
3.6.2	OTA Jalur Pertama: HiveMQ (Model <i>Push</i>)	8
3.6.3	OTA Jalur Kedua: ThingsBoard (Protokol Resmi, Model <i>Pull</i>)	9
3.6.4	Demonstrasi OTA: Pengiriman <i>firmware.bin</i> dari ThingsBoard	10
4	Kendala yang Dihadapi	12
5	Rencana Selanjutnya	13
5.1	Pengambilan Data Sensor Sungguhan	13
5.2	Pelatihan Model TinyML dengan Data Sendiri	13
5.3	Integrasi Edge Impulse	13
6	Penutup	14
6.1	Kesimpulan Sementara	14
6.2	Saran	14

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Electronic Nose (E-Nose) adalah sistem yang meniru kemampuan penciuman manusia untuk mengenali aroma melalui pola respons sekumpulan sensor gas. Proyek ini mengembangkan E-Nose untuk klasifikasi aroma kopi (*high_grade* dan *low_grade*), sebagai bagian dari Case-Based Project mata kuliah Teknologi IoT. Berbeda dari kebanyakan proyek IoT berbasis Arduino/ESP-IDF, firmware pada proyek ini diwajibkan ditulis dalam bahasa Rust dengan target `no_std` (tanpa sistem operasi), memanfaatkan pustaka `esp-hal` untuk mikrokontroler ESP32-S3. Pilihan ini menuntut penanganan manual pada aspek-aspek yang biasanya disediakan oleh framework tingkat tinggi, seperti manajemen memori, TLS, dan pembaruan firmware.

Laporan ini bersifat **laporan kemajuan (progress report)**, bukan laporan akhir. Laporan menguraikan apa yang telah berhasil diimplementasikan dan diuji hingga saat penyusunan, kendala teknis yang ditemukan beserta cara penyelesaiannya, serta rencana kerja tahap berikutnya.

1.2 Tujuan Proyek

1. Membangun firmware akuisisi data sensor pada ESP32-S3 (Rust `no_std`) dengan konektivitas ganda ke platform cloud IoT (HiveMQ Cloud dan ThingsBoard Cloud).
2. Mengimplementasikan pembaruan firmware jarak jauh (OTA) serta menyiapkan kerangka klasifikasi TinyML untuk data sensor kopi.

1.3 Ruang Lingkup Laporan

Laporan ini mencakup seluruh modul yang telah dikembangkan sampai saat ini: akuisisi sensor, konektivitas ganda, aplikasi GUI, kerangka TinyML (dengan model contoh/*demo*), serta dua jalur OTA yang berbeda. Pengambilan data sensor kopi yang sesungguhnya dan pelatihan model klasifikasi berbasis data tersebut **belum** termasuk dalam cakupan laporan ini dan akan menjadi bagian dari laporan akhir.

BAB 2

ARSITEKTUR SISTEM

2.1 Gambaran Umum

Sistem terdiri atas satu unit perangkat keras (ESP32-S3) yang membaca sensor secara lokal, lalu mengirimkan data ke dua platform cloud yang berbeda secara paralel melalui protokol MQTT. Satu aplikasi desktop berperan sebagai pengendali proses perekaman data dari sisi pengguna. Gambaran umum arsitektur adalah sebagai berikut.

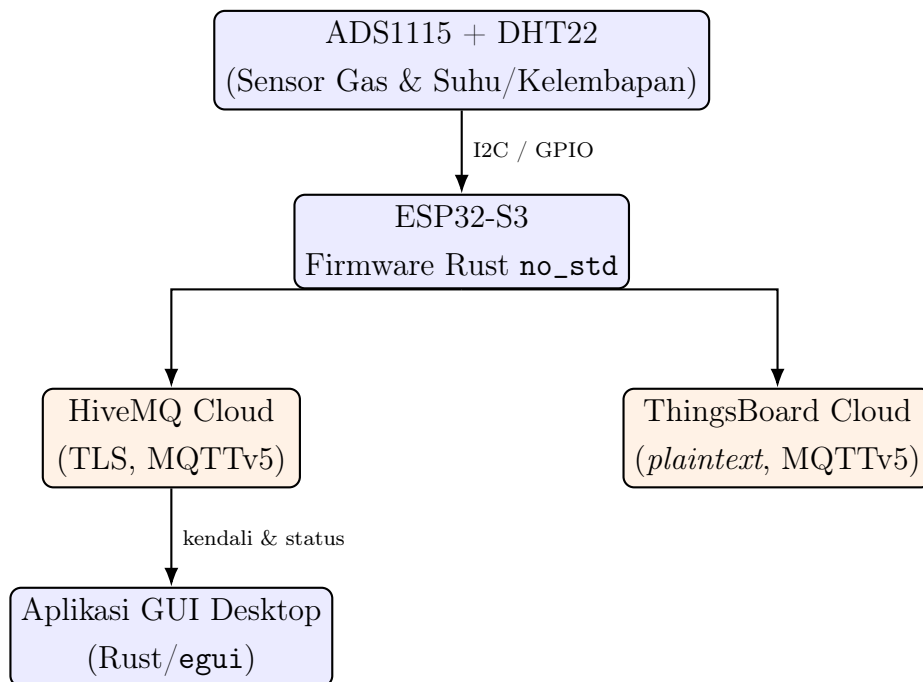


Figure 2.1: Diagram alir arsitektur sistem E-Nose

2.2 Perangkat Keras

- **Mikrokontroler:** ESP32-S3 varian N16R8 (16 MB flash, 8 MB PSRAM).
- **Sensor gas:** ADS1115 (ADC 16-bit, I2C), dirancang untuk 4–8 kanal sensor gas MOS (TGS/MQ). Pada tahap ini kanal ADS1115 masih diuji dengan tegangan referensi (GND dan 3V3), belum menggunakan sensor gas fisik.
- **Sensor lingkungan:** DHT22 (suhu dan kelembapan), terhubung pada GPIO4.

- **Indikator:** LED tambahan pada GPIO 2 sebagai penanda visual versi firmware setelah proses OTA.

2.3 Konektivitas

Firmware membuka dua sesi MQTT secara independen, masing-masing pada *task* asinkron (*embassy*) yang terpisah, sehingga kegagalan pada satu jalur tidak mengganggu jalur lainnya.

- **HiveMQ Cloud** – MQTT melalui TLS 1.2 (pustaka *mbedtls-rs*), digunakan untuk kendali perekaman dari aplikasi GUI dan status berkala.
- **ThingsBoard Cloud** – MQTT *plaintext* (tanpa TLS), digunakan untuk pengiriman telemetry suhu/kelembapan serta protokol pembaruan firmware resmi ThingsBoard.

2.4 Firmware

Firmware ditulis dalam Rust dengan target `xtensa-esp32s3-none-elf, #![no_std]`, menggunakan *executor* asinkron *embassy* di atas *esp-rtos*. Seluruh komunikasi jaringan (WiFi, TCP, TLS, MQTT) diimplementasikan langsung di atas pustaka tingkat rendah *esp-hal*, *esp-radio*, *embassy-net*, dan *rust-mqtt*, tanpa bergantung pada framework Arduino maupun ESP-IDF.

BAB 3

PROGRES YANG TELAH DICAPAI

3.1 Prototipe Perangkat Keras

Prototipe perangkat pada tahap ini terdiri atas modul ESP32-S3 (N16R8), sensor suhu dan kelembapan DHT22 yang terhubung pada GPIO4, serta sebuah LED tambahan pada GPIO 2 melalui resistor pembatas arus. LED ini berfungsi sebagai **indikator visual keberhasilan OTA**: setiap versi firmware diprogram dengan pola kedip (durasi *blink*) yang berbeda, sehingga perpindahan dari firmware lama ke firmware baru dapat dikenali langsung tanpa membuka log serial.

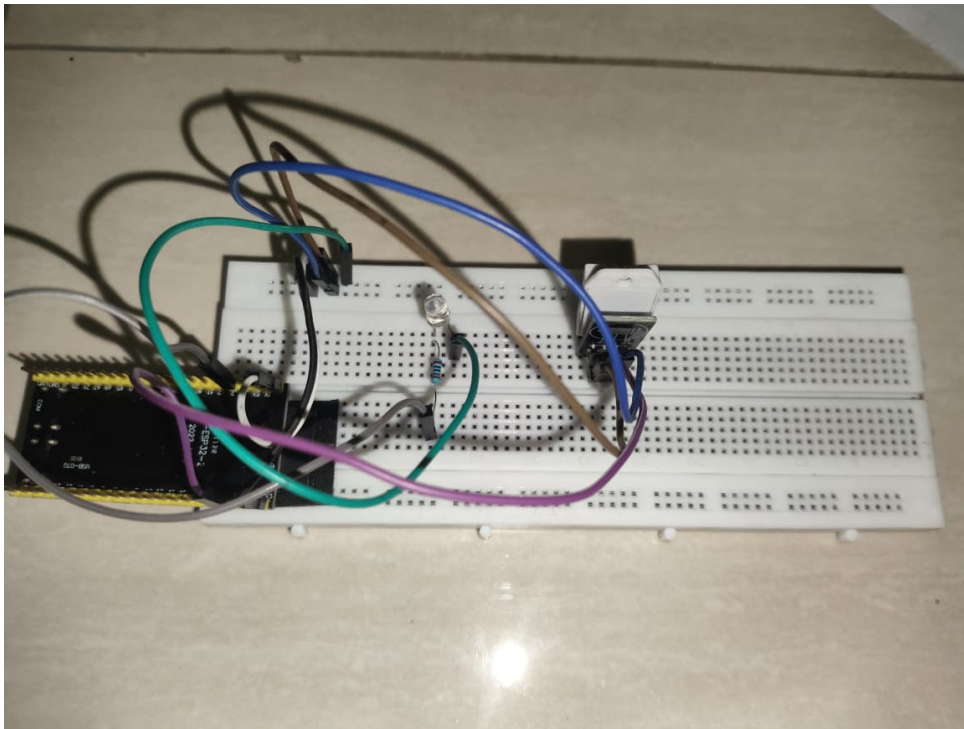


Figure 3.1: Prototipe perangkat E-Nose tahap ini: ESP32-S3, sensor DHT22, dan LED indikator OTA

Gambar 3.1 memperlihatkan susunan perangkat. ESP32-S3 berperan sebagai pengendali utama yang menjalankan firmware Rust `no_std`, DHT22 membaca suhu dan kelembapan lingkungan, dan LED menjadi penanda versi firmware yang sedang berjalan. Pada tahap ini sensor gas belum terpasang (lihat Bab 4).

3.2 Akuisisi Sensor

Modul pembacaan DHT22 diimplementasikan dengan protokol *bit-banging* manual (`critical_section`, pengukuran durasi pulsa dalam mikrodetik) karena tidak tersedia *driver* siap pakai untuk `esp-hal` pada versi yang digunakan. Modul ini telah diuji stabil, termasuk pada kondisi WiFi dan TLS aktif secara bersamaan. Modul ADS1115 ditulis langsung dari peta register datasheet (bukan menggunakan *crate* pihak ketiga), mendukung mode *single-shot*, pemilihan PGA, dan rata-rata beberapa pembacaan per kanal. Pembacaan 4 kanal (dengan tegangan referensi GND/3V3) telah diverifikasi menghasilkan nilai yang sesuai perhitungan teoritis.

3.3 Konektivitas WiFi dan MQTT Ganda

Firmware berhasil mempertahankan dua koneksi MQTT yang berjalan bersamaan sepanjang waktu operasi:

- Koneksi ke HiveMQ Cloud melalui TLS 1.2, termasuk mekanisme sambung-ulang otomatis apabila koneksi terputus.
- Koneksi ke ThingsBoard Cloud (*plaintext*), mengirimkan telemetry suhu dan kelembapan setiap 5 detik.

Verifikasi sertifikat TLS pada koneksi HiveMQ untuk saat ini masih dinonaktifkan sementara (*insecure*), sebagaimana juga diterapkan pada sketsa referensi rekan sejawat; pengaktifan verifikasi penuh direncanakan pada tahap berikutnya.

3.4 Aplikasi GUI Desktop

Aplikasi desktop dikembangkan dengan Rust dan pustaka antarmuka `egui`, berfungsi sebagai pengendali proses perekaman data dari sisi pengguna. Fitur yang telah berfungsi meliputi:

- Indikator status koneksi (broker MQTT, status ESP32 daring, kondisi sensor DHT22/ADS1115).
- Formulir parameter perekaman (nama sampel, *grade*, durasi, interval sampel).
- Tombol Mulai/Berhenti perekaman, grafik data langsung, dan tabel sampel terakhir.
- Penyimpanan otomatis ke berkas CSV dengan format kolom yang kompatibel dengan struktur dataset acuan.

3.5 Kerangka TinyML (Model Contoh)

Alur kerja TinyML telah dibangun secara menyeluruh, dari ekstraksi fitur, pelatihan model (regresi logistik ter-standardisasi), hingga eksekusi inferensi langsung pada ESP32 (`no_std`, tanpa alokasi memori dinamis). Kesetaraan hasil komputasi antara implementasi Python (untuk pelatihan) dan Rust (untuk inferensi) telah diverifikasi menghasilkan selisih probabilitas kurang dari 10^{-5} .

Catatan penting: model yang tertanam pada firmware saat ini adalah model *contoh* (*demo*), dilatih menggunakan himpunan data referensi dari pihak lain (bukan hasil akuisisi sensor pada perangkat proyek ini). Model ini ditandai secara eksplisit dalam firmware (`source = "rafi-demo"`) dan hanya berfungsi untuk membuktikan bahwa jalur inferensi berjalan dengan benar di perangkat keras yang sebenarnya. Model ini **belum** merepresentasikan kemampuan klasifikasi aroma kopi yang sesungguhnya, karena data pelatihannya bukan berasal dari sensor gas dan protokol pengukuran pada perangkat ini.

3.6 Pembaruan Firmware Over-The-Air (OTA)

Salah satu capaian teknis utama pada tahap ini adalah keberhasilan implementasi pembaruan firmware jarak jauh melalui dua jalur protokol yang berbeda, keduanya telah diuji hingga berhasil pada perangkat keras sesungguhnya.

3.6.1 Persiapan: Tabel Partisi Ganda

Firmware standar ESP32 hanya memiliki satu partisi aplikasi (`factory`). Untuk mendukung OTA, tabel partisi dirancang ulang dengan tiga partisi aplikasi (`factory`, `ota_0`, `ota_1`) beserta partisi `otadata` yang menyimpan penunjuk partisi aktif. Mekanisme ini memastikan firmware lama tetap tersimpan utuh apabila proses pembaruan gagal di tengah jalan.

3.6.2 OTA Jalur Pertama: HiveMQ (Model *Push*)

Pada jalur ini, sebuah skrip Python di sisi pengirim membagi berkas firmware menjadi potongan-potongan kecil dan mengirimkannya berurutan melalui topik MQTT khusus. ESP32 menulis setiap potongan langsung ke partisi OTA yang tidak sedang digunakan, sambil menghitung nilai CRC-32 secara bertahap. Partisi baru hanya diaktifkan apabila CRC-32 hasil perhitungan cocok dengan nilai yang dijanjikan oleh pengirim di awal transfer. Jalur ini telah diuji berhasil, termasuk pengujian pengiriman dari perangkat yang berbeda (bukan perangkat yang melakukan pemrograman awal melalui USB), membuktikan pembaruan dapat dilakukan sepenuhnya melalui jaringan internet.

3.6.3 OTA Jalur Kedua: ThingsBoard (Protokol Resmi, Model *Pull*)

Berbeda dari jalur pertama, jalur ini memanfaatkan fitur OTA bawaan platform ThingsBoard. Firmware diunggah melalui antarmuka web ThingsBoard dan ditautkan (*assign*) ke perangkat. ESP32 secara aktif meminta potongan firmware satu per satu melalui topik MQTT standar ThingsBoard (`v2/fw/request/.../chunk/...`), dan memverifikasi keutuhan data menggunakan algoritma SHA-256 (bukan CRC-32, karena ThingsBoard secara otomatis menghitung nilai SHA-256 saat pengunggahan tanpa opsi algoritma alternatif pada akun yang digunakan).

Proses pengembangan jalur ini melalui beberapa iterasi perbaikan berdasarkan hasil pengujian langsung, di antaranya:

- Perbaikan kesalahan logika pemotongan data yang menyebabkan sebagian data unduhan tidak tertulis dengan benar ke flash.
- Penyesuaian ukuran *buffer* komunikasi setelah ditemukan bahwa ukuran pesan yang diterima dapat melebihi perkiraan awal.
- Penemuan adanya batas durasi sesi (kurang lebih 5 menit) pada layanan yang digunakan, yang diatasi dengan memperbesar ukuran setiap potongan data sehingga jumlah permintaan bolak-balik berkurang dan transfer dapat selesai sebelum sesi berakhir.

Setelah perbaikan-perbaikan tersebut, jalur OTA ini berhasil diuji hingga tuntas: pengunduhan 986.560 byte firmware selesai dalam waktu kurang dari 3 menit, nilai SHA-256 tervalidasi cocok, dan perangkat berhasil melakukan *restart* otomatis menjalankan firmware baru.

```
INFO - LED ON (t=6457 ms, BLINK_MS=5000)
INFO - [ThingsBoard] Subscribe OK, minta attribute firmware. Firmware saat ini: OTA 1.0.1
INFO - [ThingsBoard] Publish OK: { "temperature": 32.0, "humidity": 70.1 }
INFO - [ThingsBoard] Firmware BARU terdeteksi: OTA 1.0.2 (994496 byte, checksum b2eefb98fdd4cf6899306c618015d5fa70a771a2f932cddfa4784fc2f1a0272f). Mulai unduh...
INFO - LED OFF (t=11458 ms, BLINK_MS=5000)
INFO - [ThingsBoard] Unduh: 16384 byte (potongan #0)
INFO - [ThingsBoard] Unduh: 32768 byte (potongan #1)
INFO - LED ON (t=16699 ms, BLINK_MS=5000)
```

Figure 3.2: Log serial ESP32: ThingsBoard mendeteksi firmware baru (versi 1.0.2) dan memulai proses unduh per potongan

Selama proses berjalan, sensor DHT22 dan indikator LED tetap berfungsi normal tanpa gangguan, sebagaimana ditunjukkan pada Gambar 3.3 yang merekam kondisi sistem pada durasi operasi berjalan (*uptime*) lebih dari 12 menit sejak dinyalakan.

```

INFO - [ThingsBoard] Publish OK: { "temperature": 32.1, "humidity": 69.4 }
INFO - LED ON (t=777213 ms, BLINK_MS=5000)
INFO - [ThingsBoard] Publish OK: { "temperature": 32.1, "humidity": 69.4 }
INFO - LED OFF (t=782213 ms, BLINK_MS=5000)
INFO - [ThingsBoard] Publish OK: { "temperature": 32.1, "humidity": 69.2 }
INFO - LED ON (t=787213 ms, BLINK_MS=5000)
INFO - [ThingsBoard] Publish OK: { "temperature": 32.1, "humidity": 69.3 }
INFO - LED OFF (t=792213 ms, BLINK_MS=5000)

```

Figure 3.3: Log serial ESP32: pembacaan sensor DHT22 dan kedipan LED tetap berjalan normal secara berkelanjutan

3.6.4 Demonstrasi OTA: Pengiriman *firmware.bin* dari ThingsBoard

Untuk membuktikan bahwa firmware benar-benar berganti, dua versi firmware disiapkan dengan pola kedip LED yang berbeda:

- **Firmware versi 1.0.0:** LED berkedip dengan periode 1000 ms (kedip cepat).
- **Firmware versi 1.0.1:** LED berkedip dengan periode 5000 ms (kedip lambat).

Berkas biner hasil kompilasi (*firmware.bin*) versi baru diunggah melalui menu *OTA updates* pada ThingsBoard Cloud, kemudian ditautkan ke perangkat E-Nose sebagai firmware target. Setelah itu ESP32 mendeteksi versi baru, mengunduh *firmware.bin* per potongan, memverifikasi SHA-256, lalu *restart*. Keberhasilan pembaruan ditandai oleh berubahnya pola kedip LED.

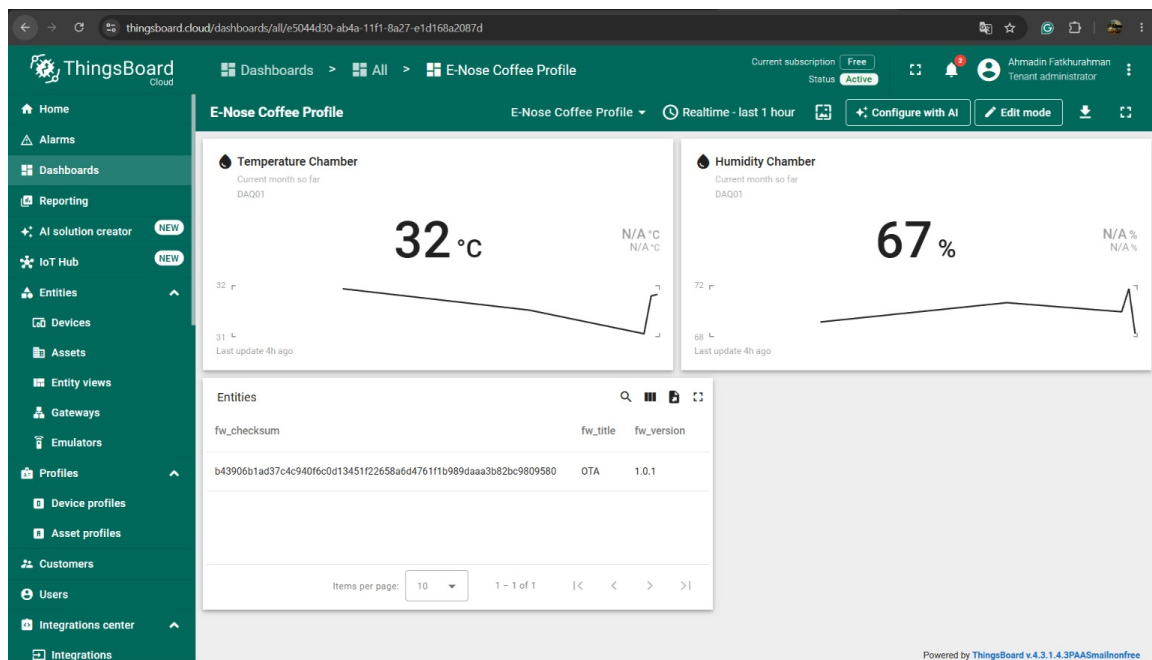


Figure 3.4: Tampilan *dashboard* ThingsBoard: data telemetry suhu dan kelembapan serta status pembaruan firmware perangkat

Gambar 3.4 menampilkan *dashboard* ThingsBoard yang memantau perangkat. Pada tampilan ini terlihat grafik suhu, grafik kelembapan, status perangkat aktif, versi firmware terkini, status OTA. Data suhu dan kelembapan dari DHT22 tetap masuk selama dan sesudah proses OTA, yang menunjukkan bahwa pembaruan tidak mengganggu fungsi akuisisi sensor.

BAB 4

KENDALA YANG DIHADAPI

1. **Keterbatasan ekosistem *crate* Rust `no_std`** untuk ESP32-S3 – beberapa pustaka pihak ketiga (misalnya untuk LED RGB *addressable* dan enkripsi TLS) memiliki masalah kompatibilitas versi atau cacat fungsional yang baru terlihat setelah diuji langsung di perangkat keras, sehingga beberapa fitur ditulis ulang langsung dari spesifikasi register/protokol tanpa *crate* pihak ketiga.
2. **Keterbatasan platform gratis (*free tier*)** – baik HiveMQ Cloud maupun ThingsBoard Cloud memiliki pembatasan tersembunyi (laju pesan dan durasi sesi) yang baru diketahui melalui pengujian berulang, bukan dari dokumentasi resmi.
3. **Ketiadaan data sensor gas sesungguhnya** – kanal ADS1115 masih diuji dengan tegangan referensi karena sensor gas fisik dan rangkaian pembagi tegangan belum dirakit, sehingga model TinyML belum dapat dilatih dengan data yang representatif.

BAB 5

RENCANA SELANJUTNYA

5.1 Pengambilan Data Sensor Sungguhan

Merancang rangkaian pembagi tegangan untuk simulasi sensor gas, dilanjutkan dengan pemasangan sensor gas fisik (TGS/MQ), serta pengambilan data mengikuti protokol pengujian (durasi 300 detik per sampel, beberapa ulangan per jenis kopi) menggunakan aplikasi GUI yang telah dikembangkan.

5.2 Pelatihan Model TinyML dengan Data Sendiri

Melatih ulang model klasifikasi menggunakan data hasil akuisisi sensor sendiri (menggantikan model contoh saat ini), serta mengevaluasi kinerja klasifikasi menggunakan skema validasi yang memisahkan data per sampel kopi (*leave-one-coffee-out*) untuk menghindari kebocoran data antar pengujian dari sampel yang sama.

5.3 Integrasi Edge Impulse

Mengeksplorasi jalur integrasi dengan platform Edge Impulse sebagai pembanding atau pelengkap model TinyML yang dikembangkan secara mandiri, termasuk mengkaji kompatibilitasnya dengan lingkungan firmware Rust `no_std` yang digunakan pada proyek ini.

BAB 6

PENUTUP

6.1 Kesimpulan Sementara

Hingga tahap penyusunan laporan ini, seluruh komponen infrastruktur inti sistem E-Nose telah berhasil diimplementasikan dan diuji pada perangkat keras sesungguhnya: akuisisi sensor DHT22 dan ADS1115, konektivitas ganda ke dua platform cloud IoT, aplikasi desktop pengendali perekaman, kerangka inferensi TinyML, serta dua jalur pembaruan firmware *over-the-air* yang berbeda pendekatan (*push* melalui HiveMQ dan *pull* melalui protokol resmi ThingsBoard). Seluruh capaian ini dikerjakan pada lingkungan Rust `no_std`, sebuah pendekatan yang lebih menantang dibandingkan penggunaan framework Arduino/ESP-IDF konvensional, namun memberikan kendali penuh atas setiap lapisan komunikasi jaringan dan manajemen memori perangkat.

Bagian yang secara sadar belum dikerjakan – akuisisi data sensor gas sesungguhnya dan pelatihan model klasifikasi berbasis data tersebut – merupakan fokus utama tahap pengembangan berikutnya, sebagaimana diuraikan pada Bab 5.

6.2 Saran

- Segera merampungkan desain rangkaian pembagi tegangan agar pengambilan data sensor dapat segera dimulai, mengingat tahap ini menjadi prasyarat bagi seluruh pekerjaan TinyML selanjutnya.
- Mempertimbangkan pengaktifan verifikasi sertifikat TLS penuh pada koneksi HiveMQ sebelum sistem digunakan di luar lingkungan pengujian.